

ASSIGNMENT-11.2

NAME: DUGYALA VINAY

HTNO: 2303A51050

BATCH: 16

Task Description -1 – (Stack Using AI Guidance)

- Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

- A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

```
1 """
2 Stack Data Structure Implementation
3 """
4 This module provides a Stack class that implements the Last-In-First-Out (LIFO)
5 data structure using a Python list as the underlying container.
6 """
7
8 class Stack:
9     """
10     A Stack implementation using a list-based approach.
11
12     A Stack is a Last-In-First-Out (LIFO) data structure where elements are
13     added and removed from the same end (top of the stack).
14
15     Attributes:
16     items (list): Internal list to store stack elements
17     """
18
19     def __init__(self):
20         """
21         Initialize an empty stack.
22
23         The stack is represented internally using a Python list. Elements are
24         appended to the end of the list (top) and removed from the end of the list.
25
26         self.items = []
27
28     def push(self, item):
29         """
30         Add an element to the top of the stack.
31
32         The push operation inserts an element at the top (end) of the stack.
33         Time Complexity: O(1) - amortized constant time
34
35         Args:
36             item: The element to be added to the stack. Can be any data type.
37
38         Returns:
39             None
40
41         Example:
42             >>> stack = Stack()
43             >>> stack.push(10)
44             >>> stack.push(20)
45             >>> stack.push(30)
46             >>> stack.items.append(40)
47
48     def pop(self):
49         """
50         Remove and return the top element from the stack.
51
52         The pop operation removes the most recently added element (LIFO principle).
53         If the stack is empty, raises an IndexError.
54         Time Complexity: O(1)
55
56         Returns:
57             The element at the top of the stack.
58
59         Raises:
60             IndexError: If the stack is empty.
61
62         Example:
63             >>> stack = Stack()
64             >>> stack.push(10)
65             >>> stack.push(20)
66             >>> stack.push(30)
67             >>> stack.pop()
68             >>> stack.pop()
69             >>> stack.pop()
70             >>> stack.items
71             []
72
73     def peek(self):
74         """
75         Return the top element of the stack without removing it.
76
77         This operation returns the top element without modifying the stack.
78         """
```

OUTPUT:

```
Popping elements:
Popped: 50, Stack: Stack (top to bottom): [40, 30, 20, 10]
Popped: 40, Stack: Stack (top to bottom): [30, 20, 10]
Popped: 30, Stack: Stack (top to bottom): [20, 10]
Popped: 20, Stack: Stack (top to bottom): [10]
Popped: 10, Stack: Stack: Empty

Attempting to pop from empty stack:
Error caught: pop from empty stack

Pushing elements and clearing:
Before clear: Stack (top to bottom): [200, 100]
After clear: Stack: Empty
```

Task Description -2 – (Queue Design)

- Task: Use AI assistance to create a Queue data structure

following FIFO principles

Expected Output:

- A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

The screenshot shows a code editor with a Python file named `task2.py`. The code implements a `Queue` class using a list-based approach. The class has methods for `enqueue`, `dequeue`, `front`, and `size`. The `enqueue` method adds an element to the rear of the queue, and the `dequeue` method removes the front element. The `front` method returns the front element without removing it, and the `size` method returns the number of elements in the queue. The code also includes docstrings and example usage.

On the right side of the screenshot, there is an AI chat interface titled "IMPLEMENTING A STACK DATA STRUCT...". The chat contains a message from the user: "Create a Queue data structure in Python following FIFO principles. Implement enqueue, dequeue, front element access, and size calculation with comments." and a response from the AI: "I've successfully created a complete Queue data structure implementation in `Assignment 11/task2.py`. Here's what's included: Core Methods:

- `enqueue(item)` - Adds elements to the rear of the queue (O(1) time)
- `dequeue()` - Removes and returns the front element with error handling
- `front()` - Views the front element without removing it (O(1) time)
- `size()` - Returns the number of elements in the queue

 Additional Features:

- `peek()` - Access the last element added

"

OUTPUT:

```
Adding tasks to queue: ['Task A', 'Task B', 'Task C', 'Task D', 'Task E']
Queue: Queue (front to rear): ['Task A', 'Task B', 'Task C', 'Task D', 'Task E']

Processing tasks (FIFO order):
Processing: Task A
Next in queue: Task B
Processing: Task B
Next in queue: Task C
Processing: Task C
Next in queue: Task D
Processing: Task D
Next in queue: Task E
Processing: Task E

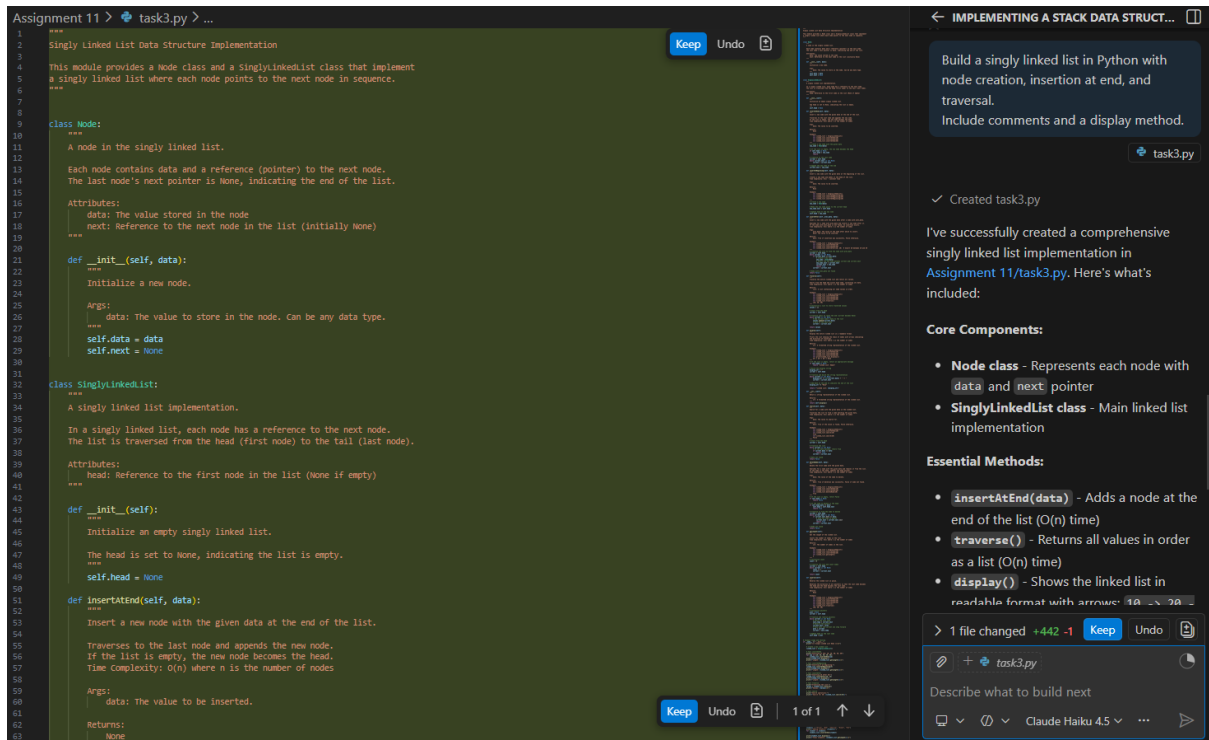
All tasks completed!
```

Task Description -3 –(Singly Linked List Construction)

- Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

- Correctly functioning linked list with node creation, insertion logic, and display functionality.



```
1  """
2  Singly Linked List Data Structure Implementation
3
4  This module provides a Node class and a SinglyLinkedList class that implement
5  a singly linked list where each node points to the next node in sequence.
6  """
7
8
9  class Node:
10     """
11     A node in the singly linked list.
12
13     Each node contains data and a reference (pointer) to the next node.
14     The last node's next pointer is None, indicating the end of the list.
15
16     Attributes:
17         data: The value stored in the node
18         next: Reference to the next node in the list (initially None)
19     """
20
21     def __init__(self, data):
22         """
23         Initialize a new node.
24
25         Args:
26             data: The value to store in the node. Can be any data type.
27         """
28         self.data = data
29         self.next = None
30
31
32  class SinglyLinkedList:
33     """
34     A singly linked list implementation.
35
36     In a singly linked list, each node has a reference to the next node.
37     The list is traversed from the Head (first node) to the tail (last node).
38
39     Attributes:
40         head: Reference to the first node in the list (None if empty)
41     """
42
43     def __init__(self):
44         """
45         Initialize an empty singly linked list.
46
47         The head is set to None, indicating the list is empty.
48         """
49         self.head = None
50
51     def insertAtEnd(self, data):
52         """
53         Insert a new node with the given data at the end of the list.
54
55         Traverses to the last node and appends the new node.
56         If the list is empty, the new node becomes the head.
57         Time Complexity: O(n) where n is the number of nodes
58
59         Args:
60             data: The value to be inserted.
61
62         Returns:
63             None
64         """
```

OUTPUT:

```
Traversing the list:
Values: [5, 10, 20, 25, 30, 40, 50]

Search operations:
Search for 30: True
Search for 100: False

Deleting node with value 25:
Linked List: 5 -> 10 -> 20 -> 30 -> 40 -> 50 -> None
Length: 6

Reversing the linked list:
Linked List: 50 -> 40 -> 30 -> 20 -> 10 -> 5 -> None
Length: 6

=== Real-World Example: Student Records ===

Adding students: ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve']
Linked List: Alice -> Bob -> Charlie -> Diana -> Eve -> None
Total students: 5

Is 'Bob' enrolled? True
Is 'Frank' enrolled? False

Removing 'Charlie' from the list:
Linked List: Alice -> Bob -> Diana -> Eve -> None
Total students: 4
```

Task Description -4 – (Binary Search Tree Operations)

- Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

- BST program with correct node insertion and in-order traversal output.

```
1 """
2 Binary Search Tree (BST) Data Structure Implementation
3
4 This module provides a TreeNode class and a BinarySearchTree class that implement
5 a binary search tree where left child values are less than parent, and right child
6 values are greater than parent.
7
8 """
9
10 class TreeNode:
11     """
12     A node in the binary search tree.
13
14     Each node contains data and references to left and right child nodes.
15     The left subtree contains values smaller than the node, and the right
16     subtree contains values larger than the node.
17
18     Attributes:
19     value (int/float): The value stored in the node
20     left (TreeNode): Reference to the left child node (Initially None)
21     right (TreeNode): Reference to the right child node (Initially None)
22     """
23
24     def __init__(self, value):
25         """
26         Initialize a new tree node.
27
28         Args:
29             value: The value to store in the node (should be comparable).
30         """
31         self.value = value
32         self.left = None
33         self.right = None
34
35 class BinarySearchTree:
36     """
37     A binary search tree implementation.
38
39     In a binary search tree:
40     - Each node has at most two children (left and right)
41     - Left child value < Parent value
42     - Right child value > Parent value
43     - This property holds for all nodes (BST property)
44
45     This structure enables efficient searching, insertion, and deletion
46     with average time complexity of O(log n) when balanced.
47
48     Attributes:
49     root (TreeNode): Reference to the root node (None if tree is empty)
50     """
51
52     def __init__(self):
53         """
54         Initialize an empty binary search tree.
55
56         The root is set to None, indicating the tree is empty.
57         """
58         self.root = None
59
60     def insert(self, value):
61         """
62         Insert a new value into the binary search tree.
63
64         """
```

OUTPUT:

```
Inserting values: [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 65]

In-Order Traversal (sorted):
[10, 20, 25, 30, 35, 40, 50, 60, 65, 70, 80]

Pre-Order Traversal:
[50, 30, 20, 10, 25, 40, 35, 70, 60, 65, 80]

Post-Order Traversal:
[10, 25, 20, 35, 40, 30, 65, 60, 80, 70, 50]

Search Operations:
Search for 40: True
Search for 100: False
Search for 25: True

Minimum value: 10
Maximum value: 80

Tree height: 3
Is tree balanced? True

=== Real-World Example: Employee ID Database ===

Adding employee IDs: [5001, 5020, 5010, 5005, 5015, 5025, 5030, 5008, 5012]
Employees in sorted order: [5001, 5005, 5008, 5010, 5012, 5015, 5020, 5025, 5030]

Employee ID 5015 found in database

ID Range: 5001 to 5030
```

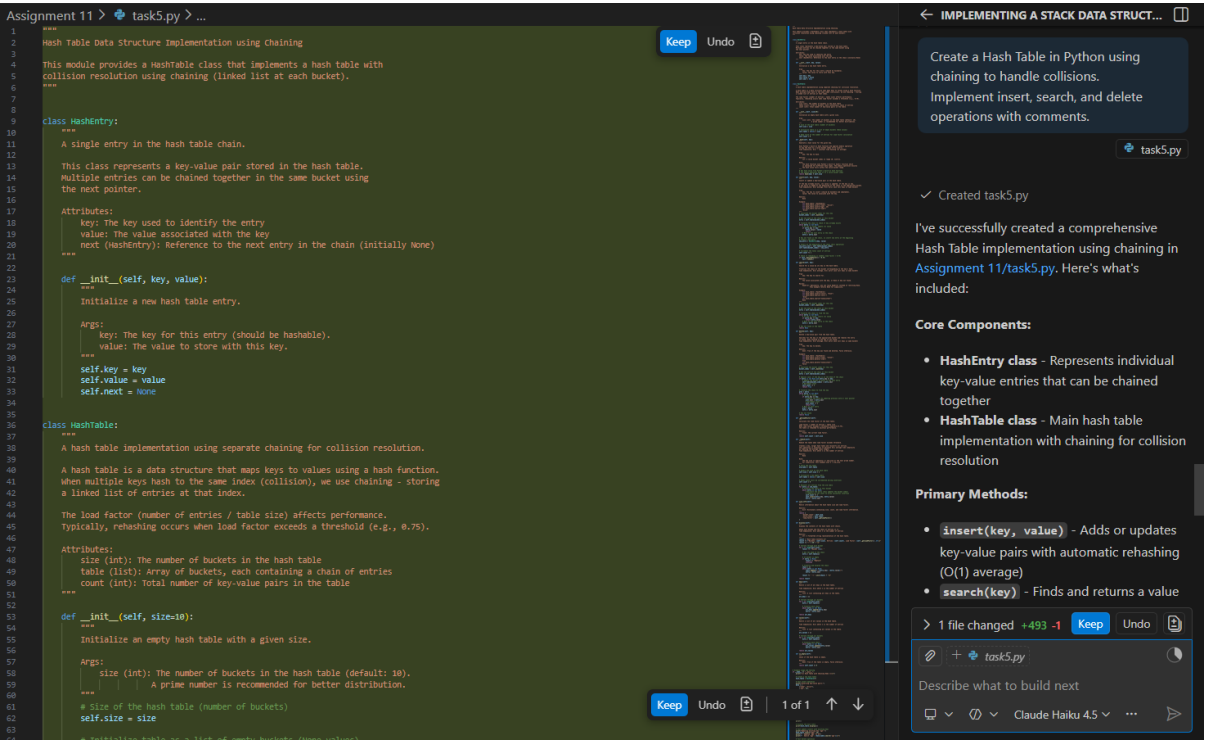
Task Description -5 – (Hash Table Implementation)

- Task: Create a hash table using AI with collision handling

Expected Output:

- Hash table supporting insert, search, and delete using chaining

or open



OUTPUT:

